

Witness-Assisted Pairing Cryptography on a Bitcoin Stack Machine

BLS12-381, Groth16 verification, and an inverted cryptographic cost model

codenlighten · built from commit cf0cc4d · tables generated from results.json

Status: preprint draft. Every number in [tables.md](#) is generated from `results.json`, which is generated by measuring the code. Prose is written by hand and can be wrong; the tables cannot silently describe a version of the implementation that no longer exists.

Abstract

We investigate whether modern pairing-based cryptography can be expressed directly in Bitcoin Script, and what it costs when it is. We compile the BLS12-381 field tower, G2 curve arithmetic, the Miller loop, the final exponentiation, multi-pairing products, subgroup membership tests and Groth16 verification into Bitcoin locking predicates built from ordinary opcodes — `OP_MUL`, `OP_MOD`, `OP_ADD`, `OP_PICK`, `OP_ROLL`, `OP_WITHIN` — with no pairing primitive, no extension-field type, and no arithmetic opcode beyond arbitrary-precision integers.

Expensive intermediate values, principally field inversions, are supplied by the spender and verified algebraically: $z = a^{-1} \pmod{p}$ becomes the constraint $a \cdot z \equiv 1 \pmod{p}$, with z bounded into $[0, p)$ so that no second witness encodes the same value.

A complete BLS12-381 pairing was evaluated on Bitcoin SV mainnet. Its two stages were first deployed and spent separately — the 63-round Miller loop as a 333,676-byte locking script and the final exponentiation as a 474,207-byte one — and then composed: a single transaction whose two inputs are the Miller loop and the final exponentiation, bound to each other by a shared output commitment, so that the network establishes the whole computation rather than its halves.

The second was made possible by a 2/6 compressed representation of the cyclotomic subgroup whose squaring is closed by construction, with a decompression derived from $f\bar{f} = 1$ as two linear equations over $\mathbb{F}_{p^2}$. It reduces the final exponentiation from 592,008 to 473,466 bytes, across the default 500,000-byte script policy boundary.

What has not executed on chain is a pairing in a *single script*: 817,031 bytes against a 500,000-byte policy. The composition moved into the transaction instead (§8.2), which is what made the result reachable at all.

We construct a Groth16 verification predicate of 1,246,986 bytes and 786,448 opcodes that checks each proof point into its prime-order subgroup, accepts a valid proof, and rejects invalid proofs whose curve membership, subgroup membership and inversion witnesses remain valid, isolating failure to the pairing equation itself. Subgroup membership is decided by endomorphism — $\varphi(P) = [-x^2]P$ on G1, $\psi(Q) = [x]Q$ on G2 — for

17,790 bytes, 1.4% of the verifier; the G2 test is nearly free because a Miller loop over Q has already computed $[x]Q$ by the time it ends (§7.6). Multi-pairing compilation reduces size by sharing the Miller accumulator, saving 609,794 bytes for two pairings and 1,223,178 for three relative to independent evaluation.

The transaction-level decomposition generalises past computations that happen to have a seam. We cut the Groth16 verifier itself into three stages across three inputs of one transaction (§8.3) — which forces a cut *inside* the Miller loop, at round 31 of 63, because three pairings sharing one accumulator are 705,838 bytes of loop alone, already over the policy before the final exponentiation is considered. The cut point is chosen by the size of the state it exposes, 2,156 bytes for a computation of 1,246,986, rather than by any structure in the mathematics. Its three coins are funded on mainnet in block 966,795 and the 1,291,329-byte transaction that spends them is mined in block 966,923 — but only after the relay path refused it for exceeding a per-transaction validation-time budget and the pool that mined it took it by direct submission (§8.3).

Correctness is established by differential testing against an independent BLS12-381 implementation on all twelve F_{p12} coefficients, adversarial mutation of every witnessed value, separation of each module's specification from its emitted schedule, randomised testing over each module's declared domain, and byte-exact reconstruction of every deployed script from source. The Groth16 verifier additionally accepts a proof generated by an independent proving stack (snarkjs), and refuses that same proof when built for a different public statement. In the course of this work the test kit was found to have been reporting as *refused* any refusal case whose spend it could not build — which included every refusal case of the whole Groth16 verifier. They have all since been run by the interpreter, and all still refuse (§10.4).

Finally, we identify an **inverted cost model**: because a witnessed inversion costs roughly four multiplications to verify rather than hundreds to compute, affine coordinates outperform the projective coordinates that conventional elliptic-curve implementation practice prescribes — and compressed cyclotomic squaring, whose decompression costs an inversion and which is therefore normally reserved for long squaring chains, becomes worth taking at once. Implementation strategies optimal for processors are not necessarily optimal for witness-assisted verification, and the same reversal appears in more than one place.

1. Introduction

Bitcoin Script is usually described as a deliberately limited language for expressing payment conditions. After the Genesis upgrade, Bitcoin SV's interpreter admits arbitrary-precision integer arithmetic, and the question of what can be *expressed* separates from the question of what is *conventional*.

This paper reports an empirical answer to a narrow version of that question: how much of a bilinear pairing can be written as a Bitcoin locking script, and what does it cost?

The contributions are:

1. **A compilation path** from BLS12-381 pairing arithmetic to Bitcoin Script primitives, through a typed, stack-tracking assembler with per-module pre- and post-conditions (§4–§6).
2. **A witness-assisted arithmetic model** in which the spender computes expensive values and the script verifies algebraic identities, with the canonicity obligation that makes this sound (§3).
3. **Empirical mainnet execution of both stages** of a pairing, as two transactions, with byte-exact reconstruction from source (§8). The final exponentiation reaches the network only because of the compression in §6.1.
4. **A multi-pairing result**: k pairings sharing one Miller accumulator, with measured savings (§7).
5. **A Groth16 verifier** and the precise separation between the values a spender may choose and the values the locking script fixes (§7.2).
6. **Subgroup membership by endomorphism**, with every fact the tests depend on computed rather than recalled, and the G2 test taken from the running point the Miller loop already produces (§7.6).
7. **Transaction-level decomposition**. A computation exceeding the script-size policy is evaluated across two inputs of one spend, with the transaction enforcing equality of the intermediate state. Given a function $f = F \circ M$ and a commitment C to the value passing between them, the technique replaces one predicate too large to relay with two that are not:

input 0: $P_1(x, s) = [s = M(x)] \wedge C(s)$ input 1: $P_2(s, y) = [y = F(s)] \wedge C(s)$

Both inputs observe the same commitment, so s is common to them, and the spend establishes $y = F(M(x))$. This is not specific to pairings: any computation with a small cut point admits it (§8.2). 8. **An inverted cost model** for elliptic-curve arithmetic under witness-assisted verification, and a mechanism that discovers it rather than asserting it (§9, §9.1).

We are explicit about what has *not* been shown. **A complete pairing has not executed as a single locking script**, and cannot at 817,031 bytes under a 500,000-byte relay policy. It has executed as a transaction-level composition of two independently verified scripts whose intermediate F_{p12} state is bound by a shared output commitment (§8.2) — a weaker statement about script size and an identical one about what was computed. The Groth16 construction verifies a statement fixed at locking time; a verifier accepting public inputs at spend time requires on-chain scalar multiplication we price but do not emit.

2. Background

2.1 The interpreter

Post-Genesis, script numbers are arbitrary precision up to 750,000 bytes and `OP_MUL` and `OP_MOD` operate at 4,096 bits. Every property this work depends on was measured rather than quoted, as passing probes.

Three behaviours shape everything below:

- **OP_MOD is truncated, not Euclidean.** $(-3) \bmod 5$ is -3 . A value that might be negative when it reaches `OP_MOD` must have the modulus added first, and this determines the schedule of nearly every module in §5.
- **OP_NUM2BIN writes a signed number**, so a high bit becomes a sign bit.
- **The stack is capped at 1000 elements**, which bounds the working set rather than the script.

2.2 BLS12-381 and the tower

BLS12-381 is a pairing-friendly curve with a 381-bit base field. The pairing $e : G_1 \times G_2 \rightarrow G_T$ satisfies $e(aP, bQ) = e(P, Q)^{ab}$, and is computed in two stages: a **Miller loop** over the 63-bit curve parameter, then a **final exponentiation** to $(p^{12} - 1)/r$.

The field tower is $F_{p^2} = F_p[u]/(u^2 + 1)$, $F_{p^6} = F_{p^2}[v]/(v^3 - \xi)$ with $\xi = u + 1$, and $F_{p^{12}} = F_{p^6}[w]/(w^2 - v)$.

Because $p \equiv 3 \pmod{4}$, the F_{p^2} Frobenius is conjugation — a single negation — which is why a degree-12 Frobenius costs seven multiplications rather than a 381-bit exponentiation (§5.4).

G_1 is the order- r subgroup of $E(F_p) : y^2 = x^3 + 4$, and G_2 the order- r subgroup of the D-type sextic twist $E'(F_{p^2}) : y^2 = x^3 + 4(u + 1)$. Neither is the whole curve:

$E(F_p) = h_1 \cdot r$ and $\#E'(F_{p^2}) = h_2 \cdot r$ with cofactors h_1 and h_2 greater than one.

3. The witness-assisted model

3.1 The transformation

Computing a modular inverse in Script would require an extended Euclidean algorithm or an exponentiation by $p - 2$. Instead the spender supplies z and the script checks

$$a \cdot z \equiv 1 \pmod{p}$$

which is one multiplication, one reduction and one comparison. In F_{p^2} it is one F_{p^2} multiplication written out — four F_p multiplications and two comparisons.

Generally: **verification can be asymptotically and practically cheaper than re-computation**, and a locking script is a verifier.

3.2 The obligation this creates

A witnessed value imposes two duties, and the second is where the bugs are.

Soundness — no *wrong* witness is accepted. $a \cdot z \equiv 1$ establishes this: there is exactly one inverse.

Canonicity — no *second* witness is accepted either. Without a range check, z and $z + p$ both satisfy $a \cdot z \equiv 1 \pmod{p}$, so a downstream comparison would be comparing numbers that are congruent rather than equal. Every witnessed value in this work is bounded into $[0, p)$ by `OP_WITHIN`.

Four bugs of exactly this kind were found by systematic audit rather than by testing, by asking of every module: *what can a spender choose, and is each choice pinned to exactly one value?* Three were in code whose tests passed.

3.3 Testing witnesses adversarially

Every witnessed input is attacked on every run: near misses, off-by- p , sign flips, zero, the modulus itself. The suite reports 1,367 forgery attempts, all refused. This is separate from testing that correct inputs produce correct outputs, and it catches a different class of defect.

4. The compilation path

```
high-level algorithm
  ↓ a JavaScript model – the specification, never the same code
field-operation schedule
  ↓ a typed, stack-tracking assembler
witness assignment
  ↓ hint(), replaying the algorithm to produce the spender's values
Bitcoin Script
  ↓ bsv.Script.Interpreter, under relay policy flags
accept / reject
```

Each module carries a model (what the answer is), an emit (how the script computes it), requires/ensures contracts as intervals, and cases. The model and the emitter are deliberately different code: a schedule and its specification must be able to disagree, or the test is vacuous.

Contracts propagate. `fp2.mul` requires its inputs reduced into $[0, p)$; when they are outputs of another module whose ensures said so, the requirement is discharged and no check is emitted. When it is not discharged, a bound is emitted. When it can be neither, the build fails. That third outcome is the mechanism's purpose.

4.1 Three sizes, named once

A module has three honest sizes and the paper reports all three, so that Table 2 and Table 5 are read as an explanation rather than a contradiction.

	what it is
core	the cryptographic computation, emitted alone
standalone	the deployable coin: core, plus an owner check and the comparison that makes it spendable
composed	the transaction-linked predicate: core, plus the covenant that publishes or consumes the intermediate state

	core	standalone	composed
Miller loop	332,977	333,676	343,799
final exponentiation	473,466	474,207	475,017

The differences are 699 bytes of coin around the Miller loop and 10,822 of covenant. Where a number appears below without a qualifier it is the **core** size, which is the one that is a property of the arithmetic rather than of how the arithmetic was wrapped.

5. Field arithmetic

See **Table 1** for measured sizes.

5.1 Fp2 multiplication, and why the products stay unreduced

Karatsuba gives $r_0 = t_0 - t_1$ and $r_1 = t_2 - t_0 - t_1$ from three products. Leaving t_0 and t_1 *unreduced* makes the imaginary part identically $a_0b_1 + a_1b_0$ — a sum of two non-negative numbers — so it needs no correction before a truncated $0P_MOD$. Only the real part is a genuine difference and pays for one reduction and one addition of p . Reducing t_0 and t_1 up front instead costs two extra $0P_MODs$ *and* still needs a $2p$ correction, because the reduced difference no longer carries the identity that makes it positive.

5.2 Cyclotomic squaring, and the basis that hid

After the easy part of the final exponentiation the accumulator lies in $G_{\Phi 6}(Fp2)$, where Granger-Scott squaring is available. Two implementations written from the nested $Fp6[w]$ indexing disagreed with general squaring and were discarded.

The third works because of a change of *presentation*, not of algebra: $w^2 = v$ and $w^6 = \xi$ mean $Fp12$ is equally $Fp2[w]/(w^6 - \xi)$ with basis $1, w, \dots, w^5$, and since $(w^3)^2 = \xi$, the element is three $Fp4$ coefficients — **(g₀,g₃)**, **(g₁,g₄)**, **(g₂,g₅)**. Those pairs are what the nested indexing conceals, and both failures had them wrong.

5.3 Sparse line multiplication

Nine of a Miller line's twelve F_{p^2} coefficients are zero. On a processor the zeros are skipped at negligible cost; in a locking script a multiplication is *bytes*, present whether or not it does anything. Writing the product against the three live coefficients takes it from eighteen F_{p^2} multiplications to fourteen.

5.4 Frobenius

$f \mapsto f^p$ is six conjugations and seven multiplications by derived constants. The module's *model* is the literal exponentiation f^p , so what is proven is that the shortcut equals the thing it is a shortcut for, rather than equalling another copy of itself.

6. The Miller loop

The loop over $|x| = 0xd201000000010000$ performs 63 squarings, 63 tangent steps, 5 chord steps and 68 line products. Two implementation points matter.

The squaring belongs to the bit, not to the line. A set bit contributes two lines and still one square. Squaring before the chord yields f^2 where f was wanted — a wrong answer costing 2,301 extra bytes to reach, which every component test tolerated.

λ is computed once. The obvious factoring derives the line from the slope and then asks a general point-addition routine for the next point, which recomputes the slope: a second inversion for a number already in hand. Deriving the next point from λ directly gives **68 inversions rather than 136**, halving the witness.

6.0 The final exponentiation as an addition chain

$\lambda = 3(p^4 - p^2 + 1)/r$ is 1,270 bits, and raising to it directly is 1,270 squarings. Two structural facts collapse it. p is a Frobenius map rather than an exponentiation, so λ written in base p becomes four digits applied by φ ; and every one of those digits is a small polynomial in the 63-bit curve parameter $y = |x|$, with coefficients no larger than 3. What remains is **5 shared ladders** — one exponentiation by y per power of y the digits need, reused across all four base- p digits — and a handful of small multiplications.

1,270 squarings become 887 with the base- p step alone, and 315 with both. The digits are derived at load time by the same arithmetic everything else uses, so a mistranscribed constant cannot occur; an earlier chain written from memory was wrong in a way that still produced an order- r element, which is to say it looked right.

6.1 Compressed squaring, and a second inverted trade

Read the Granger-Scott identities in the F_{p^4} basis — $f = c_0 + c_1w + c_2w^2$ over $F_{p^4} = F_{p^2}[s]/(s^2 - \xi)$ with $s = w^3$ — and c_1' and c_2' depend only on c_1 and c_2 . So $\{c_1, c_2\}$ is closed under squaring, c_0 can be dropped, and squaring costs two F_{p^4} squarings instead of three. That is a 2/6 compression obtained by *reading* formulas already verified rather than by deriving new ones.

Compression is only useful if one can come back, and decompression is where the technique normally stops paying: it costs a field inversion, so it is reserved for long runs of squarings. Under witness-assisted verification the inversion is four multiplications, and the break-even moves down far enough that it is worth taking immediately.

The recovery is derived rather than recalled. For f in the cyclotomic subgroup $f \cdot \bar{f} = 1$, and with $\bar{f} = \bar{c}_0 - \bar{c}_1 w + \bar{c}_2 w^2$ two of the three resulting Fp4 equations are **linear** in $c_0 = x + ys$:

$$\begin{array}{ll} w^2: & 2(c_0 \bar{c}_2)_0 = N(c_1) \quad \rightarrow \quad c_{2r} \cdot x - \xi_{c_{2s}} \cdot y = N(c_1)/2 \\ w^1: & 2(c_1 \bar{c}_0)_1 = -N(c_2) \quad \rightarrow \quad c_{1s} \cdot x - c_{1r} \cdot y = -N(c_2)/2 \end{array}$$

writing $u = u_r + u_s s$ and $N(u) = u_r^2 - \xi u_s^2$. A 2×2 solve over Fp2, one witnessed inversion of the determinant, and both equations scaled by two so nothing is halved.

	bytes
fp12.powX, uncompressed ladder	98,902
fp12.powXc, compressed	73,582
final exponentiation, before	592,008
final exponentiation, after	473,466
e(P, Q), before	935,334
e(P, Q), after	817,031

25.6% off a ladder, 20.0% off the final exponentiation, 12.6% off a pairing — and the exponentiation crosses the 500,000-byte policy boundary — which is why it is on mainnet and not only in the interpreter.

The determinant $\xi \cdot c_{1s} c_{2s} - c_{1r} c_{2r}$ can vanish, and then nothing is recovered. That is a **completeness** limit, not a soundness one: it cannot make a wrong pairing verify, only a right one fail, and the values it depends on are determined by the computation rather than chosen by the spender.

fp12.powXc is checked against the literal $f^{|x|}$ by general exponentiation — the same model fp12.powX carries — so the compressed schedule is verified against a specification no part of it produced.

7. Products of pairings, and Groth16

7.1 Sharing the accumulator

Because the squaring belongs to the bit, k pairings evaluated on one accumulator pay for 63 squarings between them and **one** final exponentiation. See **Table 3**: each pairing after the first costs about 204,000 bytes rather than 817,000.

7.2 Binding the verification key to the predicate

Groth16 verification is

$$e(A, B) \cdot e(-L, \gamma) \cdot e(-C, \delta) = e(\alpha, \beta)$$

with $L = IC_0 + \sum x_i \cdot IC_i$. **The soundness is not the arithmetic; it is who chooses what.** Only A, B and C come from the unlocking script. γ , δ and L are pushed as constants by the locking script, and $e(\alpha, \beta)$ is the constant they are compared against.

A predicate taking all three pairs as inputs is a check of the *equation* and not a verifier: a spender free to choose γ can choose one satisfying the equation for a proof of nothing. The two constructions differ by 490 bytes of pushed constant, and the test suite asserts the separation on every run.

7.3 The statement is the UTXO

L is a constant because the statement is fixed when the coin is locked. A covenant that pays out to whoever proves *this* knows its public inputs when it is written, so L is computed off chain once and pushed as 96 bytes rather than costing about 40,000 bytes of on-chain scalar multiplication per input.

This is a structural fit rather than an optimisation. In a general verifier the statement arrives with the proof; in a UTXO the statement is the output. A verifier accepting public inputs at spend time is a different object and does need the multiplications.

7.4 Fault-isolation testing

The negative cases are constructed so that failure can only be attributed to the pairing equation. For an A and a C each displaced by one generator:

$$\begin{aligned} \text{CurveValid} = 1 \wedge \text{SubgroupValid} = 1 \wedge \text{WitnessValid} = 1 \wedge \text{FieldArithmeticValid} = 1 \\ \wedge \text{Groth16Equation} = 0 \quad \rightarrow \quad \text{Script} = \text{FALSE} \end{aligned}$$

Every point remains on the curve and in its subgroup; every witnessed inverse remains correct for the points supplied, because the witness generator recomputes them. Nothing fails until the equation does. A verifier that rejects invalid proofs because they are malformed has not been shown to check the intended relation.

7.5 A proof from an independent proving stack

The fixture above is constructed from the group law, which establishes that the verifier checks the intended relation and says nothing about whether that relation is the one a real prover produces. So the verifier is also run against a proof **snarkjs** generated over BLS12-381 — its own trusted setup, its own prover, its own field arithmetic — for the circuit $c = a \cdot b$ with c public ($a = 3$, $b = 11$, so the public signal is 33).

Two things had to agree before the Script test meant anything, and neither was guaranteed. snarkjs's G2 points are $[[x_0, x_1], [y_0, y_1]]$ in the same Fp2 basis used here, with no conjugation and no swap — checked by putting the swapped reading on the twist, where it does not lie. And snarkjs checks $e(-A, B) \cdot e(\alpha, \beta) \cdot e(L, \gamma) \cdot e(C, \delta) = 1$ where

this checks $e(A,B) \cdot e(-L,\gamma) \cdot e(-C,\delta) = e(\alpha,\beta)$; they are the same equation rearranged, and the proof satisfies it under this implementation's pairing.

The locking script accepts the proof. It refuses a displaced one, points off their curves, and points on their curves outside their subgroups — and, the case worth having, **it refuses the same valid proof when the verifier is built for a different public input**. L is a compile-time constant, so a different statement is a different locking script: the statement is bound into the coin rather than supplied alongside the proof. A verifier that accepted a valid proof of something else would be checking arithmetic and not checking a claim.

A second snarkjs circuit asserts "*the holder was at least 21 years old as of 2026*" with the birth year private. Its proof is accepted; a proof generated from an underage witness — snarkjs's prover does not check constraints, so one can be produced — is refused; and the valid proof is refused by a coin that asks for 30.

`npm run groth16:external` and `npm run demo`; the vectors are in `test/vectors/groth16-bls12381/` and `test/vectors/groth16-age/` with the scripts that produced them.

7.6 Subgroup membership

A proof point that satisfies its curve equation need not lie in the order- r subgroup. $E(\mathbb{F}_p)$ has $h_1 \cdot r$ points and $E'(\mathbb{F}_{p^2})$ has $h_2 \cdot r$, and outside the subgroups the pairing is not the bilinear map Groth16's soundness argument concerns. A standard verifier checks membership; until this revision this one did not, and §10.3 recorded that as its most significant open finding.

The direct test is $[r]P = O$, a 255-bit ladder. Each curve has an endomorphism that acts on its subgroup as a small power of x , and comparing the two decides membership for much less:

$\begin{array}{ll} G1 & \varphi(x, y) = (\beta x, y), \quad \beta^3 = 1 \\ G2 & \psi = \text{untwist} \circ \pi_p \circ \text{twist} \end{array}$	$\begin{array}{ll} P \in G1 & \Leftrightarrow P \in E(\mathbb{F}_p) \wedge \varphi(P) = [-x^2]P \\ Q \in G2 & \Leftrightarrow Q \in E'(\mathbb{F}_{p^2}) \wedge \psi(Q) = [x]Q \end{array}$
---	---

Endomorphism tests for BLS12-381 are due to Bowe [12], and these two forms to Scott [13]. They are not taken from either here. Each " $\Leftarrow$ " is a short argument, and each argument rests on facts about this curve that are easy to state and easy to misremember, so `npm run subgroup` computes every one:

G1. $\varphi^3 = 1$ and $\varphi \neq 1$, so $\varphi^2 + \varphi + 1 = 0$ as an endomorphism. If $\varphi(P) = [-x^2]P$ then $O = (\varphi^2 + \varphi + 1)P = [x^4 - x^2 + 1]P = [r]P$, and since $r \nmid h_1$ the points of $E(\mathbb{F}_p)$ killed by r are exactly $G1$. The tool checks $r = x^4 - x^2 + 1$,

$E(\mathbb{F}_p) = p + 1 - t = h_1 \cdot r$ with $t = x + 1$, that $\varphi^2 + \varphi + 1 = O$ on points *outside*

$G1$, and which cube root β is — both are tried against the generator, one acts as $[-x^2]$ and the other as $[x^2 - 1]$, the two roots of $t^2 + t + 1$ modulo r .

G2. ψ satisfies Frobenius's characteristic polynomial $\psi^2 - t\psi + p = 0$. If $\psi(Q) = [x]Q$ then $O = [x^2 - tx + p]Q = [p - x]Q$, and $p - x = h_1 \cdot r$. Q also has $[h_2 \cdot r]Q = O$; $\gcd(h_1, h_2) = 1$ gives $[r]Q = O$; and $r \nmid h_2$. The tool derives ψ 's two coefficients from the untwisting map $-\xi^{((1-p)/3)}$ and $\xi^{((1-p)/2)}$, because the D-type twist untwists by division — checks the characteristic polynomial on twist points outside G2, and does not assume h_2 : a sextic twist has one of six possible orders, all six are computed, and exactly one annihilates random twist points. $\gcd(h_1, h_2) = 1$ is a property of BLS12-381, not of the family, and it is the fact the G2 test turns on.

The tool uses its own arithmetic, not the library's, and asks **@noble/curves** — which shares no code with this implementation — for its own torsion test on the generators, random multiples, $(0, 2)$, lifted points, cofactor points and mixtures. All three answers agree on every point.

The G2 test is nearly free, and the reason generalises. A Miller loop over Q starts T at Q and runs the bits of $|x|$ through it — a doubling per bit, an addition per set bit — so when the loop ends, T is $[|x|]Q$, computed by exactly the witnessed ladder a standalone test would emit. Checking B is therefore a comparison against a value the verifier already has: $T = -\psi(B)$, four congruences and one conjugate-and-scale, 389 bytes including the twist equation. The same holds for any spender-supplied G2 point that enters a pairing — σ in a BLS signature is another. We have not seen this observation stated, and have not searched for it.

It carries a precondition that no module can check about itself: T must be the loop's T , never a value the spender supplied. Both verifiers take it from their own loop, `npm test` asserts that no split stage takes in a final running point, and the fault-isolation test below gives stage 2 a B outside G2 with the check and without it.

The G1 test is two ladders by $|x|$. $|x|$ has six set bits, so $[x^2]P$ as [|x|](#) is 136 point operations, where one ladder by x^2 would be about 190 and $[r]P$ about 380. The scalar is a compile-time constant, so no step branches. `g1.inSubgroup` is 8,517 bytes with its 136 witnessed inverses, and it contains the curve equation for a reason worth stating: short-Weierstrass doubling and addition never read b . Given coordinates off $y^2 = x^3 + 4$, the ladder runs on whichever $y^2 = x^3 + b'$ they lie on — a curve with its own ϕ and its own group order, on which the argument above also goes through. Without the curve check the module would certify order r on a curve nobody asked about.

	bytes
<code>g1.inSubgroup, standalone</code>	8,517
<code>g2.inSubgroup, standalone</code>	389
the Groth16 verifier without subgroup checks	1,229,196
with them	1,246,986
the checks	17,790 — 1.4%

The previous revision priced these as ladders at 5–10% of the verifier. The G2 half turned out to be a comparison, and the G1 half cheaper than a ladder by the choice of scalar.

Which check refuses is tested, not inferred. The whole verifier refuses a proof with A outside G1 for two reasons at once — the subgroup test, and a pairing equation that no longer holds — so the whole verifier cannot say which check did the work. A split stage can (§8.3): stage 1 checks no equation at all. Stages 1 and 2 are each built twice, with the subgroup checks and without, and given $A + (0, 2) - (0, 2)$ has order 3, so this is on the curve and of order $3r$ — and $B + [r]R$ for a twist point R. Without the checks both stages accept and carry the point forward; with them both refuse.

The verifying key is checked once, off chain, when the coin is built: a key with $\alpha, \beta, \gamma, \delta$ or any IC_i outside its subgroup does not compile.

8. The mainnet experiment

See **Table 5** and **Table 7**.

The complete Miller loop was deployed as a 333,676-byte locking script and spent. One $f \mapsto f^{|x|}$ ladder was deployed as 99,631 bytes and spent. Each deployed script is rebuilt from source and compared byte for byte against the chain; **21 of 21 reconstruct exactly**, a count generated from the ledger rather than written down. One of them, the pairing chain rebuilt with scriptmin-minimized stages, reconstructs from source *and* the scriptmin commit its ledger entry records; the rest from source alone. The two-way split of §8.2 is rebuilt coin by coin; before this revision its record was checked against the chain for size only, which the count above did not say.

Scope. A complete pairing has executed on the network, as two inputs of one transaction (§8.2):

funding	92bb3f0e790ace19f1afec51141f87a3d6a911f10e82e765dba5368a6765667c
spend	fd0f553ee9a96b2cb48a4a9712824580910cd6056fc42f93f49eae83da2fb9e2

The stages were also deployed and spent individually beforehand. Neither figure includes `fp12.powX`, a correct uncompressed ladder the network executed and which §6.1 has since taken off the critical path; counting a superseded version would inflate the result.

What has *not* executed is a pairing in a single script, which at 817,031 bytes is past the policy and always will be without a further 39% reduction. The claim is "a complete pairing has been evaluated on mainnet, across two inputs of one transaction" — the qualifier is load-bearing and is not dropped anywhere in this paper.

8.1 Systems findings

Scripts of this size exposed two wallet defects that had never mattered at a few hundred bytes.

Coin selection inherits ancestry. Outputs were taken in indexer order, which swept up dust from earlier deployments — and that dust sits at the *end* of the unconfirmed chain those deployments created. A single fresh 50,000-satoshi output funds a 334 KB

transaction alone; funded by three satoshis of dust first, the same transaction inherited fourteen links of ancestry and was rejected with `too-long-mempool-chain`. The fee was never the problem.

Fee estimation must be script-derived. Funding stopped at a fixed amount + 5000 constant — reasonable for a 400-byte covenant, meaningless for a script whose fee alone is 33,389 satoshis. It did not underfund; it swept the wallet, which is how the first defect acquired its dust.

Both are ordinary bugs. We report them because they are the kind of assumption that only very large scripts falsify, and anyone repeating this work will meet them.

8.2 Decomposing across a transaction

A pairing in one script is 817,031 bytes against a 500,000-byte policy, and the gap is not closable by optimisation. It is closable by putting the composition somewhere other than the script.

```
input 0  the Miller loop           publishes f as an OP_RETURN output
input 1  the final exponentiation  consumes that same output
```

Both inputs of a spend observe the same `hashOutputs`. If each requires that commitment to be the data output *it* constructs, the two must have constructed identical bytes — enforced by the transaction rather than assumed. Neither script contains the other's code, which is the necessary property: a covenant can only commit to a successor whose bytes it can build, and a 344 KB script cannot carry a 475 KB one.

The argument is short enough to give in full. Write *M* for the Miller loop, *F* for the final exponentiation, and *C(s)* for "the transaction's output set is exactly the data output carrying *s*". The two predicates are

```
P1(x, s) = [ s = M(x) ] ∧ C(s)
P2(s, y) = [ y = F(s) ] ∧ C(s)
```

A transaction spending both inputs satisfies *P₁(x, s₀)* and *P₂(s₁, y)* for the *s₀* and *s₁* each input used. *C* is a function of the transaction alone, so *C(s₀)* and *C(s₁)* hold of the same output set; since *C* determines *s* (the output *is s*, and `hashOutputs` is collision-resistant), *s₀* = *s₁* = *s*. Therefore

```
y = F(s) = F(M(x)) = e(P, Q)
```

and the transaction's validity is exactly the statement that the pairing was computed correctly. Nothing in the argument mentions pairings: it holds for any *f* = *F* ∘ *M* with a cut point whose state is small enough to publish.

	lock	unlock	verdict
input 0, pairing.publish	343,799	350,922	accepted
input 1, pairing.consume	475,017	479,302	accepted

	lock	unlock	verdict
output, OP_RETURN	588	—	twelve Fp12 coefficients

Both inputs are verified by `bsv.Script.Interpreter` under relay policy flags, **and the transaction is on mainnet**: funding `92bb3f0e...`, spend `fd0f553e...`, 164,997 satoshis. What the network establishes, in one transaction, is that the Miller loop ran correctly on P and Q, that its twelve outputs were published, that the same twelve were consumed, and that their final exponentiation is $e(P, Q)$.

Joint grinding. `OP_PUSH_TX` requires each preimage to satisfy a canonical low-S condition, ordinarily obtained by grinding that input's `nSequence`. But `hashSequence` commits to *every* input's sequence, so grinding input 0 invalidates input 1's preimage and conversely: independent grinds do not compose. The search is over the pair. About one preimage in fifty passes, so a pair lands in a few thousand attempts — 733 in the reported run. Rebuilding a preimage carrying a 344 KB `scriptCode` that many times is unnecessary: only 36 bytes change, so each preimage is constructed once and `hashSequence` and `nSequence` are patched in place, then checked against the ones the library produces.

A constraint this surfaces. `OP_PUSH_TX`'s preimage *contains the script it unlocks*, so a covenant's unlocking script is as large as its locking script. `pairing.consume` locks in 475,017 bytes and unlocks in 479,302 — 20,698 under the policy. For covenants of this size the binding limit is the **unlocking** script, not the locking one, which is not where one would look for it.

What the shared commitment does not establish. The argument above is sound about the bytes and silent about the input set: $C(s)$ constrains what the transaction publishes, not which inputs are in it. A stage spent alone satisfies its covenant vacuously, and for P_2 that is exploitable rather than merely untidy. F is exponentiation by $d = 3(p^{12}-1)/r$; the target $e(P, Q)$ has order r ; $\gcd(d, r) = 1$. So $f = e(P, Q)^{(d^{-1} \bmod r)}$ satisfies $[y = F(f)]$ for two modular exponentiations and no Miller loop. We carry this out against the interpreter and the coin is accepted. No value is transferred — the covenant forbids change — but a reader who checks one input concludes a computation occurred that did not, and an adversary can consume the coin ahead of the honest spend and prevent the real one from ever being published.

The remedy is the field the construction never read. `hashPrevouts` is the double-SHA of every spent outpoint in order, each a 32-byte `txid` followed by a 4-byte little-endian index. The stages of one computation are outputs of a single funding transaction, so the entire list is a function of that one `txid`: witness 32 bytes, rebuild the list in script, require the hash to equal the preimage's `hashPrevouts`, and require this input's own outpoint — at offset 68 — to be the slot the script claims. The predicate becomes

$$P_i(s_{\{i-1\}}, s_i) = [s_i = F_i(s_{\{i-1\}})] \wedge C(B) \wedge S_i$$

with S_i asserting that the spend has exactly N inputs, all outputs of one transaction at indices $0..N-1$, and that this is input i . It costs 47 bytes at two stages and 56 at three.

What remains outside the script is that those outputs carry the scripts they are supposed to — an output point does not name a script — which is one check on one funding transaction rather than a judgement about the spend. All three stages of §8.3 bind their siblings; the deployed two-way instance predates the finding and is reported with it.

This generalises past pairings. Any computation too large for one script can be cut at a point where the intermediate state is small, with each piece requiring the same commitment to that state. The state here is 588 bytes for a computation of 817,031, and the cut is where the two stages of the pairing already meet.

8.3 Three ways, and a cut with no natural seam

The two-way decomposition cut a pairing where its own structure already divided it. A Groth16 verifier does not offer that convenience. It is 1,246,986 bytes, and its Miller loop — three pairings already sharing one accumulator — is

705,838 bytes

on its own. That is 205,838 over the policy *before* the final exponentiation is considered. Cutting at the M/F boundary yields a piece that still does not fit, and no rebalancing of two stages can fix it: the loop must itself be cut.

It is cut at round 31 of 63. Nothing distinguishes round 31 — it is not a seam in the mathematics, only the point at which the two halves of the loop are each small enough. What makes an arbitrary cut admissible is that the state there is small: the accumulator f , twelve F_p coefficients, and the running point T of each of the three pairings, four coefficients apiece. Twenty-four values.

The subgroup checks of §7.6 are placed where each is cheapest. Stage 1 checks A and C into G_1 before its half of the loop. B cannot be checked there — at round 31 its running point is $[|x| \gg 32]B$, not $[|x|]B$ — so stage 2 checks it after the loop ends. The running point stage 2 *resumes* from is a witness, but the blob pins it to the one stage 1 computed, so the one it finishes with is $[|x|]B$.

	lock	unlock	verdict
input 0, $A, C \in G_1$; rounds 1-31; publishes S_1	399,402	424,186	accepted
input 1, rounds 32-63; $B \in G_2$; publishes S_2	371,834	383,299	accepted
input 2, final exponentiation vs $e(\alpha, \beta)$	475,647	481,527	accepted
output, <code>OP_RETURN</code>	2,156	—	44 field elements

The subgroup checks account for 16,849 bytes of stage 1 and 957 of stage 2. Each stage bounds only the blob's values at its entry — the proof and the two cut states, which are compared as bytes — and leaves every inverse to the module that reads it; the audit checks by provenance that every numeric witness of every stage is still bounded (§10.3).

All three are verified by `bsv.Script.Interpreter` under relay policy flags. The transaction is 1,291,329 bytes and establishes, as a single atomic object, that a Groth16 proof of the statement "*the holder was at least 21 years old as of 2026*" — generated by `snarkjs`, not by us (\$7.5) — verifies, with its points in their subgroups.

And it is on mainnet. `bin/deploy-groth16.js` built the funding transaction and the spend with the same code that produced the figures above, ran all three inputs through the interpreter against the real funding txid, and only then broadcast them:

```
funding 025f20f1d4156aa354afef37b1ec67e5c461f11375bc739902845a3b985dc5f0
spend   cad6d2cca44fffb2f445009f392b668382d79d120b1c14db0fd4a8d192204bb6
```

The funding transaction is 1,247,999 bytes and carries the three coins at outputs 0, 1 and 2; the spend is 1,291,329 bytes and paid 129,133 satoshis, 253,934 for the two together at 100 sat/kB. Its `nLockTime` grind landed at 216,991.

The funding transaction was mined at once, in block 966,795. **The spend was not**, and how it eventually reached a block is the most useful thing in this section.

It was accepted into WhatsOnChain's node's mempool and went no further. Five later blocks from five different miners left it out; a second indexer never saw it; and submitting it to GorillaPool's ARC returned `REJECTED` with the node's reason, `too-long-validation-time`. It stayed unmined for 128 blocks. The operator's explanation, when asked, was that their node gives a peer-relayed transaction at most a second to validate — `maxnonstdtxvalidationduration`, 1000 ms by default — and this one exceeded it under load. Measured in the JavaScript interpreter, its three inputs take 4.5 seconds between them against 3.0 for the two inputs of the \$8.2 spend, which relayed and was mined: 1.5 times as long, with no input individually heavier than \$8.2's. The pool then submitted it to their own node directly, and mined it in **block 966,923**.

So the binding constraint moved a third time, and changed kind. The script-size policy bound the single script, which is why the computation was split; the unlocking script's size bound each stage, because `OP_PUSH_TX`'s preimage contains the script it unlocks; and validation time binds the *transaction* — which is exactly the unit the decomposition put all three stages into.

Unlike the first two, this one is not a property of the transaction alone. The spend is valid, and the block containing it is valid; what refused it was the budget a node applies to work arriving from a peer. A transaction may therefore be consensus-valid, relayable nowhere, and mined anyway by an operator willing to take it by hand. We report it as mined because it is, and record how, because "it confirmed" would leave out the part a reader would need: at this validation cost, propagation cannot be relied on. A construction that does not depend on anyone's goodwill has to spread the stages across transactions rather than inputs, so that no single transaction carries more than a second of validation — a different covenant, with its own soundness argument.

The soundness argument, for N stages. Write the computation as $f = F_N \circ \dots \circ F_1$, let s_i be the state after stage i , and let B be the single data output carrying every s_i at once. Stage i 's predicate is

$$P_i(s_{i-1}, s_i) = [s_i = F_i(s_{i-1})] \wedge C(B)$$

where $C(B)$ is again "the output set is exactly the data output B ". Each stage computes its own transition and *witnesses* every other state in B . A witness is free to be wrong; what removes the freedom is that C is a function of the transaction alone, so all N inputs see the same B , and B determines every s_i . Stage i 's witnessed s_{i-1} is therefore the same s_{i-1} that stage $i-1$ *computed* and was checked against. Composing along the chain gives $s_N = f(s_0)$, and the transaction's validity is that statement.

The N -stage case differs from $N = 2$ in one respect worth stating: each stage carries the *whole* blob, not merely its own two endpoints, because C commits to the output in full. This is why the blob is kept small — 44 field elements, 2,156 bytes, for a computation of 1,246,986 — and why the cut point is chosen by the size of the state it exposes rather than by the sizes of the pieces alone.

Triple grinding, and choosing the right field to grind. One preimage in fifty is canonical (measured: 2.01%), so a pair lands in $\sim 2,500$ attempts and a **triple in $\sim 123,000$** . The two-way construction ground `nSequence`; at three inputs that becomes untenable, and for a reason specific to BIP-143. `nSequence` appears *twice* in a preimage: as its own field near the end, and inside `hashSequence` at offset 36. Moving it changes bytes 36 onward, so every SHA-256 block of a ~ 400 KB preimage must be recomputed — 123,000 times, three preimages each.

`nLockTime` appears **once**, eight bytes from the end. Grinding it instead leaves every SHA-256 block but the last unchanged, so each input's midstate is computed once and copied per attempt, and the per-attempt cost falls from hashing 400 KB to hashing 64 bytes. The reported run tried 168,127 `nLockTimes` in **2.5 seconds**; 3,451 cleared input 0, 62 cleared inputs 0 and 1, one cleared all three. Small `nLockTime` values are block heights already past, so the transaction remains final — the search stops well short of the chain height, and the deployment refuses a result that is not below it. The fast filter is checked against the library's own canonicalisation on every candidate it considers before being trusted.

This is a case of the paper's general theme arriving somewhere unexpected. The question "which transaction field should carry the nonce?" has no answer in cryptography and no answer in isolation; it is answered by where the field sits relative to a hash function's block boundaries, which is exactly the kind of target-specific fact §9.1 argues should be delegated to a search rather than decided in advance.

8.4 One stage per transaction, because relay is the binding constraint

§8.3's spend could not travel, and the remedy follows from what refused it. Validation time bounds the TRANSACTION, so the unit of work has to become the unit of relay: one stage per transaction, with the state passed along a chain of small coins rather than between the inputs of one.

The carrier coin holds a PAIR of states, prev and cur, and requires its successor to be the same script with **its own cur as the successor's prev**, leaving that successor's cur unconstrained. The stage coin beside it requires the successor's cur to be what it computed from the prev it consumed:

```
carrier:  output = body || myCur || X      X witnessed, free
stage:    output = body || p      || F(p)   p witnessed, free
```

Both hash the same output, so $p = \text{myCur}$ and $X = F(p)$. Neither reads the other's script — an outpoint does not name one — and they agree because the bytes they each rebuild are the same bytes.

	lock	unlock	validates in
tx ₁ , the Miller loop	344,840	351,963	1,229 ms
tx ₂ , the exponentiation and the carrier	476,067	480,352	1,834 ms
§8.2's two-input spend, which relayed	—	—	3,031 ms
§8.3's three-input spend, which did not	—	—	4,539 ms

The heaviest link asks for 0.60 of what relayed and 0.40 of what did not. The milliseconds are one idle run on one machine, recorded in `relay.json` beside the processor that produced them; a node's C++ is far quicker, and what transfers is the ratio rather than the number. It is on mainnet: funding 2b7ec0a5..., then a8ae58b2... and 0482ec-cb..., all three mined in block 966,954 — by the pool whose relay check had refused §8.3's spend, and which minutes earlier refused this chain's first link for paying two satoshis under its size-derived minimum. That was our arithmetic error, since fixed; what it records about the platform is that a node's relay threshold, a miner's mining threshold and consensus are three different things.

What it gives up is atomicity. In §8.3 every stage checked through `hashPrevouts` that the spend consumed exactly its siblings, and the transaction's validity was the whole statement. Here the carrier arrives from the previous transaction, whose txid nothing could know when the stage coins were written, so no stage can insist the carrier beside it is genuine. The claim becomes one about a chain, and a reader completes it by walking one: the funding transaction's outputs carry the stage scripts, and each link spends the coin and the carrier it should. `npm run verify:chainwalk` performs exactly that walk against the chain, and compares the value the last carrier holds with the pairing computed here.

Headroom. Stage 3 unlocks in 481,527 bytes — 18,473 under the policy, again in the unlocking script rather than the locking one. A four-way split would relieve it, and the construction admits one without modification.

8.5 The verifier itself, as a chain

§8.4 chained a pairing. The construction that refused to relay was a Groth16 VERIFIER, and the same treatment applies to it: three stages, one per transaction, carried by the coin of §8.4.

	lock	unlock	transaction	relative
tx ₁ , rounds 1-31, A and C ∈ G ₁	401,185	425,348	428,996	0.48×
tx ₂ , rounds 32-63, B ∈ G ₂	368,525	379,961	388,967	0.40×
tx ₃ , the final exponentiation	477,044	482,307	491,313	0.58×

Relative is against the §8.2 spend that relayed and was mined. The heaviest link asks for 0.58 of it and 0.39 of the §8.3 spend that relay refused, so every transaction here travels on the same terms as one that already has. The largest unlocking script is 482,307 bytes, 17,693 under the policy — 780 bytes MORE than §8.3's stage 3, because the 1,568-byte state push costs slightly more than the 44 numeric pushes it replaces. A fourth link was not needed, and would be the remedy if it were.

What changes is not the arithmetic but who chooses the state. In §8.3 all three stages wrote the same output, and sibling-binding forced stage 1's *witnessed* S₂ to equal stage 2's *computed* S₂. Across transactions there is no shared output and no sibling, so a link that merely witnessed its predecessor's state would prove nothing. Links 2 and 3 therefore take the state as bytes and SPLIT IT OUT of what the carrier beside them holds: stage 2 no longer witnesses the proof or S₁, stage 3 no longer witnesses S₂. The 32 and 44 spender-chosen field elements of §8.3 become one carrier-pinned field.

The carrier's width follows from that. Link 2 needs the proof and S₁, 32 elements; link 3 needs only S₂, 12. One width serves the chain, so it is the wider of the two — 1,568 bytes — rather than the full 44-element blob, which would have cost stage 3 another 1,176 bytes in the script with the least room.

Where a bad proof dies, and that it matters where. A chain is not atomic, so the question is not whether it refuses but at which link:

	dies at
A on the curve and outside G ₁	link 1, the subgroup check
a valid proof of a different statement	link 3, against e(α, β)

Both refusals are NUMEQUALVERIFY from the check named. The early links of a losing run are valid transactions and will confirm; what no spender reaches is the last carrier.

The binding, attacked by mechanism. The carrier enforces prev and the stage enforces cur, and an attack that refuses through the wrong one has tested nothing — so

each names the input that must reject it. Handed a state the carrier is not holding, the stage ACCEPTS (it computed honestly from what it was given) and the carrier refuses. Handed a cur the stage did not compute, the carrier ACCEPTS (it constrains only prev) and the stage refuses. An earlier version of this test was built without grinding its preimage and was refused by OP_PUSH_TX before either mechanism was consulted; it is recorded here because a refusal for the wrong reason is the defect of §10.4 in another hat.

And the limitation, run rather than asserted. A carrier from another execution, beside a stage consistent with it, is accepted — it is a valid link of THAT chain. No script can distinguish the two: an outpoint does not name a script, and the carrier's txid did not exist when the stage coins were written. The tool builds that splice and reports the acceptance, rather than describing the caveat in prose and hoping nobody tries it.

`npm run groth16:chain` is the whole of the above.

It is on mainnet. Funding 1c0b9af1, then 15c6e1ab, f93163d2 and 3b68669c — all four mined in block 966,988, the first block after they were broadcast. Every link relayed normally: no direct submission, no operator taking anything by hand, which is the entire difference from §8.3. The deployment cost 255,644 satoshis against §8.3's 253,934, so the chain is within one per cent of the transaction that could not travel.

`npm run verify:groth16chainwalk` is the reader's side of it, and it is not a formality: it rebuilds all three stage scripts from source and finds them byte for byte in the funding transaction's outputs, follows each link to the coin and the carrier it should spend, and compares the state the last carrier holds with $e(\alpha, \beta)$ computed here. Thirteen checks, because the chain cannot make them for itself.

9. An inverted cost model

Conventional elliptic-curve implementation assumes $I \gg M$ and chooses projective or Jacobian coordinates to eliminate inversions, accepting several extra multiplications per step.

Under witness-assisted verification, an F_{p^2} inversion is one F_{p^2} multiplication and two range checks. Empirically $I_{\text{witnessed}} \approx 4M$, so the trade reverses:

affine > projective

The Miller loop here is affine throughout, and projective formulas would make it larger. We state this as the paper's most transferable claim:

The optimal cryptographic algorithm depends on the verifier's computational model. A witness-assisted stack machine has a different cost hierarchy from a processor, and received implementation wisdom does not transfer unexamined.

Three further consequences of the same principle appear above. Multiplying by zero is free on a processor and costs bytes in a script (§5.3). An LSB-first ladder's multiplication-by-one is free at runtime and costs 3,110 bytes when unrolled — the marginal cost

of an Fp12 product inside a composition (§5, Table 1). And compressed cyclotomic squaring, whose decompression inversion normally confines it to long squaring chains, pays immediately here (§6.1) — the same reversal, arrived at independently.

9.1 Instruction selection: letting the target decide

Stating "affine beats projective in Script" is a finding about one construction. Making it *operational* is a different thing, and the repository does: given several modules that compute the same mathematical function, `select()` proves they agree and then chooses by a stated objective over the cost vector (locking bytes, unlocking bytes, total, opcodes, peak stack depth, fee).

$f \mapsto f^2$ in Fp12	lock	unlock	total	opcodes	stack
fp12.sqr	2,292	588	2,880	1,540	37
fp12.cycSqr	1,259	588	1,847	968	34 ◀
$f \mapsto f^{ x }$ in Fp12					
fp12.powX	98,902	588	99,490	71,954	61
fp12.powXc	73,582	1,175	74,757	54,204	71 ◀

Nothing there was told that the cyclotomic identities or the compression win. Both were emitted and counted. This is the smallest honest form of an instruction-selection pass over mathematics: the same function, several lowerings, and a target-specific cost model choosing among them — which is what turns §9's observation from an assertion into a mechanism.

The total column is where a witnessed construction can lose. `fp12.powXc` buys 25,320 bytes of locking script for 587 bytes of unlocking script; a construction with a worse ratio would be rejected on the same evidence, and the objective is a flag rather than an assumption.

The problem being solved

Stated formally, the selector solves

$$P^* = \operatorname{argmin}_{P \in \mathcal{P}(f, D)} C(P)$$

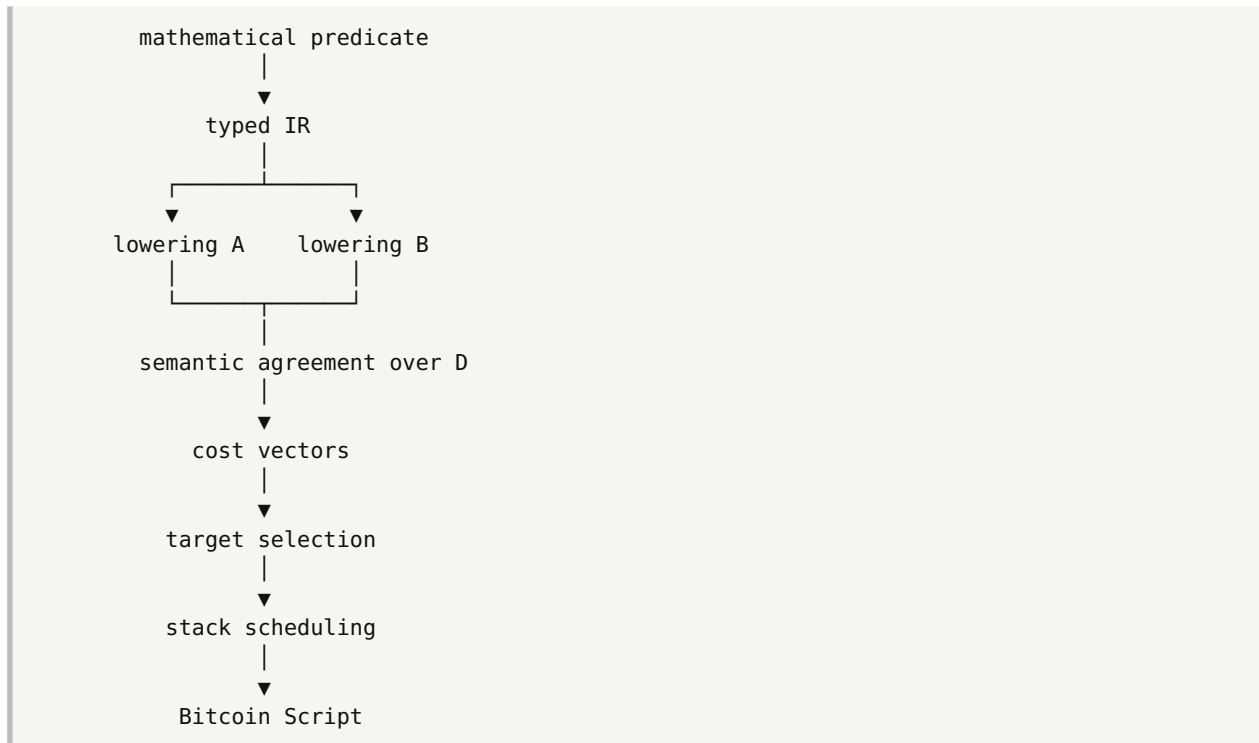
where $\mathcal{P}(f, D)$ is the set of Script programs *proven* equivalent to the mathematical function f over the domain D , and C is the measured cost vector (locking bytes, unlocking bytes, total, opcodes, peak stack depth, fee). Both qualifiers carry weight. The domain D is part of the problem, not a footnote: `fp12.cycSqr` belongs to $\mathcal{P}(\text{square}, G_{\Phi 6})$ and not to $\mathcal{P}(\text{square}, \text{Fp12})$, and a selector that ignores D returns a wrong answer confidently. And membership of $\mathcal{P}$ is *established*, not assumed — see below.

Witnesses widen $\mathcal{P}$ rather than merely changing costs within it. A program need not compute f ; it need only satisfy

$$\exists w : P(x, w) = 1 \Leftrightarrow f(x) = 1$$

so a verification relation cheaper than any evaluation of f is admissible. The witnessed inverse is exactly that: $a \cdot z \equiv 1 \pmod{p}$ with z supplied is not a cheaper way to compute an inverse, it is a different program that certifies one. This is why the cost model inverts (§9) rather than merely shifting — the feasible set is not the same set.

The pipeline this implies is a compiler back end:



Everything above the dashed middle exists here in embryo: the modules are the lowerings, requires/ensures are the typed IR's facts, `select()` is the agreement-and-choice pass, and the assembler is the scheduler. What is not yet built is a front end that takes a formula and produces the lowerings, and a scheduler that does register allocation rather than following the order a human wrote.

The agreement step is the contribution, not the ranking. A selector that picks the cheaper of two implementations without first establishing that they compute the same thing is a benchmark attached to a bug. And comparing their *models* is insufficient — we found this by trying it. `fp12.cycSqr`'s model is the general square, correct on all of Fp12 ; only its emitted script is confined to the cyclotomic subgroup. Two candidates whose models agree can have scripts that do not. The check therefore verifies each candidate's script against its own model on every vector, and the models against each other:

```
script_i = model_i = model_j = script_j
```

Run off the subgroup the same two candidates are refused rather than ranked, and that refusal is itself a test case. Agreement is always agreement *on a domain*, and each contest records the one its vectors came from; "cheaper" without "and equivalent, here" is not a result.

10. Correctness methodology

1. **Independent reference.** The BigInt implementation agrees with @noble/curves on all twelve Fp12 coefficients — not merely on bilinearity, which a wrong pairing can also satisfy. An early line function here was non-degenerate, produced order- r values, passed every component test, and was wrong; byte equality caught it. Eight chosen scalar pairs, three boundary pairs (1 and $r-1$ in each position), and twelve generated from a printed seed: 320 assertions in all.
2. **Model versus emitter.** Each module's specification is different code from its schedule.
3. **The real interpreter**, under relay policy flags.
4. **Adversarial witnesses**, on every run: 1,367 forged witnesses refused.
5. **Published vectors** where they exist (RFC 6238, BIP-340), and the host library's own secp256k1 and Merkle implementations.
6. **Randomised differential testing of every module** against its own model, on inputs generated from the module's stated domain rather than chosen: 47 modules, none disagreeing. The four modules whose domain is a *subgroup* — which an interval cannot state — carry generators that produce elements of it, so those rounds test the module where it is defined rather than reporting a disagreement that is the module telling the truth.
7. **Boundary values:** 0, 1, $p - 1$, and both coefficients at $p - 1$.
8. **Byte-exact chain reconstruction** of every deployed script, coin by coin for deployments of several coins.
9. **Generated figures.** Every number in Table 1–8 is produced by measurement.
10. **Computed number theory.** The facts the subgroup tests rest on are computed by `npm run subgroup` with arithmetic independent of the library's, and the tests' verdicts are checked against an independent implementation's (§7.6).
11. **Refusals by the interpreter.** A refusal case is counted only when the script refused; a case whose spend cannot be built is a failure (§10.4).

10.1 Reproducibility of the numbers, not only of the artifact

Three documents in this repository quoted measured figures in prose and all three had drifted — one claimed an RSA verifier was 955 bytes when it was 969, another that a curve addition was 139 bytes when a soundness fix had made it 167.

Measurements are therefore generated:

```
source → measurement → results.json → tables → this paper
```

rather than `source → someone remembers → a number in a PDF`. The test suite fails if `results.json` disagrees with the code, and if `tables.md` disagrees with `results.json`.

The figures for the three-way split were the exception until this revision: copied into `results.json` from a run of the tool that produced them, and therefore regenerated by nothing. They are now produced by building the spend inside the generator and verifying all three inputs before measuring it.

10.2 Related work

A note on this section. The references below are the standard results this implementation is built on and are cited because it uses them. The *positioning* claim — that lowering pairing arithmetic to a general-purpose script language's integer and stack primitives has not been done this way before — has not been established by a systematic literature search, and is stated as an impression rather than a result. It must be settled before submission.

Pairings and their computation. Miller's algorithm [1] is the loop; the BLS family of pairing-friendly curves is due to Barreto, Lynn and Scott [2], and BLS12-381 in particular to Bowe [3], now specified in the CFRG's pairing-friendly-curves draft [4]. The loop used here is the optimal ate pairing [5]. The final exponentiation's cyclotomic structure follows Granger and Scott [6], and the compressed representation of §6.1 is a variant of the compression Karabina introduced [7] — arrived at here by reading [6]'s identities in the $\mathbb{F}_{p^4}$ basis rather than by following [7], and differing from it in which two coefficients are dropped.

Subgroup membership. Endomorphism-based membership tests for BLS12-381 are due to Bowe [12]; the forms $\varphi(P) = [-x^2]P$ and $\psi(Q) = [x]Q$ used in §7.6 are Scott's [13]. The conditions they rest on were computed for this curve rather than taken from either.

The proof system. Groth16 [8] is the verifier of §7.

Implementation cost models. The projective-coordinate practice that §9 inverts is standard throughout the pairing-implementation literature; Beuchat et al. [9] and Aranha et al. [10] are representative of the optimisation strategies assumed there, all of which take a processor's $I \gg M$ as given.

The platform. Bitcoin Script's transaction-introspection technique depends on the BIP-143 sighash structure [11], which post-Genesis Bitcoin SV retains and whose arbitrary-precision arithmetic makes any of this possible.

What is not cited, and should be. Verifier implementations on other blockchain virtual machines — Ethereum's alt_bn128 precompiles, zkSync, and the various Solidity Groth16 verifiers — occupy the adjacent space and rely on native pairing operations or on a VM designed with the arithmetic in mind. A proper comparison against them belongs here and has not been written.

- [1] V. S. Miller. The Weil pairing, and its efficient calculation. *Journal of Cryptology*, 17(4), 2004.
- [2] P. S. L. M. Barreto, B. Lynn, M. Scott. Constructing elliptic curves with prescribed embedding degrees. SCN 2002.
- [3] S. Bowe. BLS12-381: New zk-SNARK elliptic curve construction. 2017.
- [4] Y. Sakemi, T. Kobayashi, T. Saito, R. S. Wahby. Pairing-Friendly Curves. IRTF CFRG Internet-Draft.
- [5] F. Vercauteren. Optimal pairings. *IEEE Transactions on Information Theory*, 56(1), 2010.
- [6] R. Granger, M. Scott. Faster squaring in the cyclotomic subgroup of sixth degree extensions. PKC 2010.
- [7] K. Karabina. Squaring in cyclotomic subgroups. *Mathematics of Computation*, 82, 2013.
- [8] J. Groth. On the size of pairing-based non-interactive arguments. EUROCRYPT 2016.
- [9] J.-L. Beuchat et al. High-speed software implementation of the optimal ate pairing over Barreto–Naehrig curves. Pairing 2010.
- [10] D. F. Aranha et al. Faster explicit formulas for computing pairings over ordinary curves. EUROCRYPT 2011.
- [11] P. Wuille. BIP-143: Transaction Signature Verification for Version 0 Witness Program. 2016.
- [12] S. Bowe. Faster subgroup checks for BLS12-381. IACR ePrint 2019/814.
- [13] M. Scott. A note on group membership tests for G1, G2 and GT on BLS pairing-friendly curves. IACR ePrint 2021/1130.

10.3 An adversarial audit of our own assumptions

`npm run audit` asks the four questions a reviewer would ask, structurally where possible, and reports what does not hold as prominently as what does. Five of its findings changed the implementation.

A stage was not bound to its siblings, and an attack follows. The audit's third section asks whether two inputs can disagree about f , and the answer is no. The question it was not asking is whether one input can proceed without the other, and the answer to that was yes: `hashOutputs` is checked and `hashPrevouts` is not. Section 8.2 gives the forgery — $f = e(P, Q)^{(d^{-1} \bmod r)}$, two `modexps` — and `npm run attack:siblings` runs it against the interpreter, where the coin is accepted. The fix reads `hashPrevouts` for 47 bytes at two stages and 56 at three, and is used by every stage of §8.3. The instance already on chain cannot be changed and is reported as it stands; the demonstration is kept as a standing test rather than retired once fixed, because what it protects is a property of the *transaction*, which no single script can assert alone.

This one is worth dwelling on as a methodological point. The soundness argument in §8.2 is correct — we did not misprove it. It establishes what it says, which is a statement about bytes, and we read it as a statement about the computation. Every test we had ran the honest two-input transaction, so nothing exercised the gap. The audit found it only once it was asked a question phrased in terms of what an adversary omits rather than what they substitute.

Witness canonicity, exhaustively. The test kit attacks witnesses, and samples once a module has 136 of them. Sampling cannot establish "every witness is bounded"; only structure can. Each stack slot now carries the input it descends from through picks, rolls and renames, and every bound records that origin, so the question is answered for

all **1,970 numeric witnesses** — the library's and every stage of the three-way split — at once rather than for a sample. The audit grants a range only to inputs that are not witnesses, so a witness's bound has to come from something the script does.

The first run reported `ec.add` and `ec.double` as unbounded. They were not: they bounded their witnessed inverse with a hand-written `OP_WITHIN` that the recorder could not see. That is worth stating because the fix was not to special-case them — a bound nothing can observe is a bound an audit reports as missing, so they were rewritten to go through the same call everything else uses, which emits identical bytes and is visible. Every deployed script still reconstructs byte for byte. Bounds the framework places from a module's requires are recorded the same way.

A gap in the attack generator, found the same way. Chasing that false positive exposed a true one: the kit's default attacks look for the modulus under `params.n`, and the elliptic-curve modules call it `p`. So the *same-residue* attacks — the ones that test canonicity rather than soundness, and the ones the earlier four bugs needed — were silently skipped for every `ec.*` module. Those modules had been passing an attack suite that never asked them the question. Fixed, and they pass it now.

The proof points were not checked to be on their curves, and then not checked to be in their subgroups. The first was fixed in the previous revision at a few hundred bytes. The second stood as this audit's most significant open finding, priced at 5–10% of the verifier and left open on the grounds that the endomorphism tests had to be derived and verified rather than recalled. §7.6 is that derivation. The audit now checks that β and ψ 's constants are in the locking script, that A and C each carry a whole G1 ladder, that each subgroup module accepts a subgroup point and refuses the same point shifted by a cofactor element, and that a verifying key with a point outside its subgroup does not compile.

10.4 A refusal that never reached the interpreter

A refusal case asserts that a module rejects an input whatever the spender supplies. The kit built such a case's unlocking script from the values the case named, inside the same `try` as the evaluation — so a case that named the proof and not its witnesses threw while *pushing* an undefined value, and the throw was reported as a refusal.

Every refusal case on the whole Groth16 verifier named only the proof. So "a displaced A is refused", "a point off the curve is refused", "a proof from someone underage is refused" and "a valid proof is refused by a coin for a different statement" had been reported, in `tools/groth16.js`, `tools/groth16-external.js` and the demonstration, without the interpreter ever running them. Cases on smaller modules were unaffected because they pin their witnesses, which is why the defect survived: every refusal it hid was one that was also expected to be true.

It was found while adding the subgroup cases, by reading the kit rather than by a test failing. The kit now fills a refusal case's unnamed witnesses from the module's hint — which is the stronger test anyway, every inverse correct for the points supplied so that nothing fails until the property does — and a case that still cannot be built is a failure.

A ninth deliberately broken module in the kit's self-test is a refusal case that omits a witness for an input the module accepts, and the kit is required to report it. Every affected case has since been run by the interpreter, and every one still refuses.

11. Limitations

- **A complete pairing has not executed on mainnet in a single script** and cannot at 817,031 bytes under a 500,000-byte script policy. It has executed across two inputs of one transaction, which is a different and weaker statement about script size and an identical one about what was computed.
- **The three-way Groth16 spend did not propagate.** It is mined (§8.3), but the relay path refused it for its validation time and a pool took it by direct submission; nothing about it should be read as evidence that a transaction of this validation cost will reach a miner by itself.
- **fp12.powX is deployed but superseded.** It computes a correct $f^{|x|}$ and the network executed it; fp12.powXc replaced it, so it is not a component of the pairing measured here and its bytes are not counted as such.
- **The Groth16 verifier is for a fixed statement.** Spend-time public inputs need on-chain scalar multiplication, priced at about 40,000 bytes per input but not emitted.
- **The G2 subgroup test has a precondition no module can check about itself:** that the running point it compares against is the Miller loop's own. It is tested structurally and by fault isolation (§7.6), not proven from inside the module.
- **Groth16's own soundness is out of scope**, as is the setup that produced a verifying key. The verifier is tested against proofs from an independent proving stack (§7.5) and against a fixture built from the group law (§7.4); neither says anything about whether Groth16 is sound, only about whether this checks its verification equation on points of the right groups.
- **One proving stack, two circuits.** Nothing here depends on circuit size — the verifier's cost is ℓ public inputs and three pairings regardless — but the claim rests on proofs from one prover.
- **fp12.cycSqr is correct only on the cyclotomic subgroup**, a precondition the interval-based contract system cannot express. It is discharged structurally by the easy part, and stated as a note rather than enforced.
- **Policy, not consensus.** The 500,000-byte boundary is a relay-policy number.
- **One implementation of the compiler.** The measurements are of this emitter; a better one would produce smaller scripts.
- **The novelty claim is unestablished.** §10.2 says what this is built on. It does not say what has been done before, because that has not been searched properly, and no priority claim should be read into its absence.

12. Roadmap

The final exponentiation is 57.9% of a pairing and is still where remaining effort belongs, though §6.1 has taken it below the policy boundary. Better addition chains, fur-

ther witness-assisted subcomputations and transaction-level decomposition across several outputs are all open. Whether a whole pairing can be brought under the boundary in one script is an engineering question, not an expressiveness one. Bringing the whole composition under the 500,000-byte boundary would turn the existing transaction-level pairing into a **single-script** one, removing the need for inter-input state binding and simplifying deployment to a single coin. That is a further 317,031 bytes, and it is the remaining milestone rather than the remaining question.

Beyond BLS12-381: BLS signature verification is a two-pairing product; KZG opening is likewise. Both land directly on the multi-pairing predicate in §7.1, and in a BLS signature the spender's G2 point σ enters a Miller loop, so its subgroup test costs a comparison (§7.6). The public key is a G1 point fixed in the script and checked when the coin is built.

Further out, §9.1 is the seed of something larger. If a target-specific cost model can choose between two lowerings of a squaring, it can choose between lowerings of anything, and the interesting object stops being any particular pairing implementation and becomes the compiler: a mathematical predicate in, a locking script out, with the choice of representation, the placement of witnesses and the stack schedule all decided by measurement against this target rather than inherited from a processor's. The results here suggest that such a compiler would make materially different choices from a conventional one, and that those choices are discoverable rather than needing to be known in advance.

13. Conclusion

Bitcoin Script has no pairing opcode, no extension-field type, and no inversion primitive. It has arbitrary-precision multiplication and modular reduction, and a complete BLS12-381 pairing was nonetheless evaluated by the network — through two transaction-bound predicates, one computing the Miller loop and publishing its Fp12 result, the other consuming that committed result and performing the final exponentiation. Each stage was also deployed and spent on its own. A Groth16 verifier for a fixed statement is 1,246,986 bytes with its proof points checked into their subgroups, validated against the interpreter on proofs from an independent prover, and evaluated by the network across three inputs of one transaction, which reached a block only because a pool accepted it directly. The prices are measured rather than argued about.

The general lesson is not about pairings. It is that shifting computation into prover-supplied witnesses, and verifying algebraic identities instead of recomputing values, changes which algorithm is optimal. The absence of an opcode is not the absence of the computation; it is a cost, and costs can be measured.

Artifact

Source, generator, test vectors, deployed scripts, transaction identifiers and a one-command reproduction path accompany this paper.

```
npm test          # everything, including one whole pairing in the interpreter
npm run pairing:prove # emit  $e(P, Q)$ , run it, check twelve coefficients
npm run subgroup    # compute the facts the subgroup tests rest on
npm run groth16      # emit the verifier: one valid proof, six refused
npm run groth16:split # three stages, the subgroup checks isolated, one transaction
npm run paper        # regenerate results.json and tables.md from the code
npm run verify:chain  # rebuild every deployed script and compare to the chain
```

Appendix — the tables, generated from results.json

Table 1 — the field tower, as Bitcoin Script

Every module below is proven against `bsv.Script.Interpreter` under relay policy flags at the BLS12-381 prime.

Two columns, because a module has two honest sizes and neither is *the* size.

Standalone is the module alone: it pushes the 381-bit prime itself, 49 bytes, which a locking script has to do once however many operations follow.

Marginal is the same module with the prime already supplied as a stack value. For the small modules that is much smaller — `fp2.mul` is 39 rather than 90, a difference larger than the module. For the large ones it is slightly *larger*, and the reason is worth stating: a prime pushed by the module's own prologue sits directly above its inputs, while one supplied beneath twenty-four of them is deeper, and an `OP_PICK` past index 16 costs a byte more. Neither column is a lower bound on the other; the depth of a constant is part of its cost.

module	operation	standalone	marginal	opcodes
<code>fp2.mul</code>	Karatsuba, three Fp multiplications	90	39	30
<code>fp2.sqr</code>	two Fp multiplications	77	22	22
<code>fp2.add</code>		64	13	14
<code>fp2.sub</code>		68	18	18
<code>fp2.neg</code>		63	14	14
<code>fp2.conj</code>	the Fp2 Frobenius, $p \equiv 3 \pmod{4}$	59	9	10
<code>fp2.mulXi</code>	multiply by $\xi = u + 1$, no multiplications	67	15	16
<code>fp2.inv</code>	witnessed: $a \cdot a^{-1} = 1$, both bounds	101	51	36
<code>fp6.mul</code>	Karatsuba, six Fp2 multiplications	756	721	520
<code>fp12.mul</code>	Karatsuba, eighteen Fp2 multiplications	3,073	3,110	2,036
<code>fp12.sqr</code>	general squaring	2,292	2,301	1,540
<code>fp12.cycSqr</code>	Granger-Scott, cyclotomic subgroup only	1,259	1,292	968
<code>fp12.mulLine</code>	sparse: nine of twelve coefficients are zero	2,051	2,018	1,310
<code>fp12.frob</code>	$f \mapsto f^p$, six conjugations and seven products	715	675	330
<code>fp12.inv</code>	witnessed: one Fp12 product and twelve bounds	3,431	3,414	2,166
<code>fp12.powX</code>	$f \mapsto f^{ x }$, MSB-first, 63 squarings and 5 products	98,902	101,353	71,954
<code>g2.stepDouble</code>	tangent at T, and 2T	552	503	375
<code>g2.stepAdd</code>	chord through T and Q, and $T + Q$	535	487	363

module	operation	standalone	marginal	opcodes
g1.onCurve	$y^2 = x^3 + 4$	72	21	17
g2.onCurve	$y^2 = x^3 + 4(u + 1)$	178	127	102
g1.inSubgroup	witnessed: the curve, and $\varphi(P) = [-x^2]P$ by two ladders	8,517	9,321	6,032
g2.inSubgroup	the twist, and $T = -\psi(Q)$ for the loop's $T = [x]Q$	389	340	144

Cyclotomic squaring is 43.9% smaller than general squaring, and the sparse line product is 35.1% smaller than a general $F_{p^{12}}$ product.

Table 2 — the pairing

stage	composition	bytes	opcodes	witnessed values
Miller loop	63 tangents, 5 chords, 68 line products, 63 squarings	332,977	215,308	136
final exponentiation	easy part, 5 ladders, 15 terms, 19 Frobenius maps	473,466	323,175	72
e(P, Q)	the two, chained	817,031	538,743	208

The stages do not sum to the whole: chaining them costs 10,588 bytes that neither separate measurement contains.

Table 3 — products of pairings share the accumulator

The squaring belongs to the *bit*, not to the line, so k pairings on one Miller accumulator pay for 63 squarings between them and one final exponentiation.

pairings	as a product	as separate pairings	saved
1	817,031	817,031	—
2	1,024,268	1,634,062	609,794
3	1,227,915	2,451,093	1,223,178

Each pairing after the first costs about 203,647 bytes rather than 817,031.

Table 4 — a Groth16 verifier

$e(A, B) \cdot e(-L, \gamma) \cdot e(-C, \delta) = e(\alpha, \beta)$, with 2 public inputs fixed at lock time.

locking script	1,246,986 bytes
opcodes	786,448
witnessed values	752
chosen by the spender	$Ax, Ay, Bx_0, Bx_1, By_0, By_1, Cx, Cy$

chosen by the locking script	γ , δ , L , and $e(\alpha, \beta)$
subgroup checks, $A, C \in G1$ and $B \in G2$	17,790 bytes, 1.4%
the same verifier without them	1,229,196 bytes

The row naming what the locking script chooses is the soundness property. A spender who could choose γ could choose one satisfying the equation for a proof of nothing. The subgroup rows are the other half of it: a point outside its subgroup is still a point, and the pairing is not bilinear on it.

Table 5 — what the network has executed

	bytes	share of a pairing	status
Miller loop, complete	332,977	40.8%	mainnet
final exponentiation, complete	473,466	57.9%	mainnet
$e(P, Q)$ in one script	817,031	100%	interpreter
fp12.powX, superseded	98,902	—	mainnet

A complete BLS12-381 pairing has been evaluated by the Bitcoin network, as two inputs of one transaction:

funding	92bb3f0e790ace19f1afec51141f87a3d6a911f10e82e765dba5368a6765667c
spend	fd0f553ee9a96b2cb48a4a9712824580910cd6056fc42f93f49eae83da2fb9e2
input 0	pairing.publish, 343,799 bytes
input 1	pairing.consume, 475,017 bytes
the spend	830,928 bytes, one OP_RETURN output

The network verified that the Miller loop ran correctly on P and Q , that its twelve outputs were published, that the same twelve were consumed, and that their final exponentiation is $e(P, Q)$.

What has **not** run on chain is a pairing in ONE SCRIPT. That is 817,031 bytes against a 500,000-byte policy, and the 10,588 bytes by which the whole exceeds its two stages are the plumbing the transaction replaced.

The last row carries its own caveat. fp12.powX — one $f^{|x|}$ ladder, uncompressed — was deployed and spent and is correct, and it is no longer what the implementation does: fp12.powXc replaced it at 73,582 bytes. Counting its bytes toward a pairing on chain would be counting a version that no longer exists, so the figure above does not.

Compressed squaring is what made the second row possible: the final exponentiation was 592,008 bytes before it and is 473,466 after, which is the difference between over the policy boundary and under it.

Table 6 — field operations per pairing

Counted by running a pairing that agrees with an independent implementation on all twelve Fp12 coefficients. Fp2 counts include the operations the Fp12 routines perform.

	Fp2 mul	Fp2 sqr	Fp2 add	Fp2 sub	Fp2 inv	Fp12 mul	Fp12 sqr	Fp12 cycSqr	Fp12 line
Miller loop	1,912	131	3,056	2,436	68	0	63	0	68
final exponentiation	1,306	3,087	11,536	4,614	1	63	0	342	0

Table 7 — mainnet deployments

21 deployments, each rebuilt byte for byte from the source and compared against the chain by `npm run verify:chain`. The count is generated from the ledger, so this table and the prose cannot disagree about how many there are.

module	locking script	deploy
vault	433	0b3055d52367223d...
rsa.verify	981	37c0c7a710905482...
tx.locktime ▶ totp.verify	746	15dfa4c4ea56ad73...
ecdsa.verify	59,247	2e2830799509ba4f...
sha256.block	49,240	4cbc7f96da81f787...
schnorr.verify	59,721	28d106a6cff81c70...
rsa.verify ▶ tx.hash0Outputs	1,386	186ff18e5e480b90...
ecdsa.verify	59,297	fe4184c05bbd3d41...
tx.transition ▶ state.counter	470	eb58479abf8ab19a...
tx.transitionPaying ▶ state.limit	510	3226e448dc8a5c51...
fp12.sqr ▶ fp12.mulLine	5,269	1bf828187c39d0b5...
pairing.miller48	257,558	d5395e02a492a7af...
pairing.miller63	333,676	10c52d6dfb2831ec...
fp12.powX	99,631	1caac568ad58e904...
pairing.finalExp	474,207	038dbe94167beee2...
pairing.publish ▶ pairing.consume	818,816	92bb3f0e790ace19...
groth16.stage1 ▶ groth16.stage2 ▶ groth16.stage3	1,246,883	025f20f1d4156aa3...
slhdsa.spend	75,324	44db75c3235ea1aa...
pairing.chainMiller ▶ pairing.chainExp	820,907	2b7ec0a583168c15...
groth16.chain1 ▶ chain2 ▶ chain3	1,246,754	1c0b9af1f1b39d40...
pairing.chainMiller ▶ pairing.chainExp, minimized by scriptmin	504,058	7948b2a31e07484b...

21 of 21 reconstruct exactly.

Table 8 — a Groth16 verifier across three inputs of one transaction

The three-pairing Miller loop alone is 705,838 bytes, past the policy, so the loop itself is cut, at round 31 of 63. Built by `src/groth16spend.js` and every input verified before these figures were taken.

input	computes	locking	unlocking	of which subgroup checks
0	$A, C \in G_1$; rounds 1–31	399,402	424,186	16,849
1	rounds 32–63; $B \in G_2$	371,834	383,299	957
2	final exponentiation against $e(\alpha, \beta)$	475,647	481,527	—
output	the blob, 44 field elements	2,156	—	—

The transaction is 1,291,329 bytes. Its three `OP_PUSH_TX` preimages were ground together over `nLockTime`, 168,127 tries for a triple. Every stage binds its siblings through `hashPrevouts`, 56 bytes apiece.

On mainnet: funding `025f20f1d4156aa354afef37b1ec67e5c461f11375bc739902845a3b985dc5f0`, spend `cad6d2cca44fffb2f445009f392b668382d79d120b1c14db0fd4a8d192204bb6`. How the spend reached a block — refused by the relay path for its validation time, and taken directly by the pool that mined it — is §8.3.